

Implementação de uma Rede em Chip SPIN com Topologia Fat-Tree de 8 Roteadores em SystemC

Olive Oliveira Medeiros
Departamento de Matemática Aplicada
Universidade Federal do Rio Grande do Norte
Natal, RN, Brasil
olivemedeiros68@gmail.com

Yasmin Giordano Santos
Departamento de Matemática Aplicada
Universidade Federal do Rio Grande do Norte
Natal, RN, Brasil
giordanoyasmin06@gmail.com

Resumo—Este trabalho apresenta a concepção e a implementação, em SystemC, de uma Rede em Chip (NoC) baseada na arquitetura SPIN (*Scalable, Programmable, Integrated Network*). A topologia adotada é uma *Fat-Tree* de dois níveis com oito roteadores e oito *hosts*, organizados em quatro roteadores folha (conectados diretamente aos *hosts*) e quatro roteadores raiz (responsáveis pela interligação entre subárvores). Cada roteador foi modelado de forma granular, com submódulos independentes de *buffer* de entrada (*FifoBuffer*), lógica de roteamento (*RoutingLogic*) baseada em divisões e comparações aritméticas sobre o identificador de destino, árbitro *Round-Robin* e matriz de comutação (*crossbar*), todos sincronizados por um protocolo de *handshake valid/ready*. Três cenários de tráfego foram simulados para cobrir os extremos do espaço de latência da rede: o pior caso, com três saltos entre subárvores distintas (80 ns); o cruzamento entre subárvores distintas, também com três saltos (80 ns); e o melhor caso, com destino local sem necessidade de escalar a hierarquia (40 ns). Em todos os cenários, os pacotes injetados alcançaram o *host* de destino correto, com integridade total dos dados e latência determinística, validando o algoritmo de roteamento, o mecanismo de arbitragem e o controle de fluxo implementados.

Index Terms—Rede em Chip, NoC SPIN, *Fat-Tree*, SystemC, Roteamento, Round-Robin, Handshake, FIFO, Crossbar.

I. INTRODUÇÃO

Este relatório tem como objetivo cumprir o requisito avaliativo da disciplina DIM0129 — Organização de Computadores, ministrada pelo Prof. Dr. Márcio Eduardo Kreutz, referente à implementação de um modelo de Rede em Chip (NoC) SPIN com oito roteadores. O trabalho é acompanhado pelo código-fonte disponibilizado em <https://github.com/minkyzecapagods/spin-noc-8routers> e pelo presente relatório.

O crescimento contínuo da complexidade dos circuitos integrados tornou o barramento compartilhado tradicional um gargalo arquitetural relevante: à medida que o número de núcleos de processamento e módulos de memória em um único chip aumenta, o barramento único passa a ser disputado por um número excessivo de agentes, o que degrada a largura de banda efetiva, eleva o consumo de energia e compromete a escalabilidade do sistema. Nesse contexto, as Redes em Chip (*Networks-on-Chip*, NoC) consolidaram-se

como a solução arquitetural dominante para a comunicação intrachip em sistemas multiprocessados modernos [1]. Uma NoC substitui o barramento por uma infraestrutura de comunicação estruturada, composta por roteadores autônomos interligados por enlaces ponto a ponto segundo uma topologia específica, permitindo que múltiplas transferências ocorram simultaneamente em caminhos distintos da rede [2].

Dentre as topologias propostas na literatura, a SPIN (*Scalable, Programmable, Integrated Network*) destaca-se por sua estrutura em *Fat-Tree*, que garante caminhos redundantes entre os nós e distribui o tráfego entre os roteadores de nível superior [3]. Nesse modelo, os roteadores dividem-se em dois grupos: os roteadores folha, conectados diretamente aos *hosts*, e os roteadores raiz, responsáveis por interligar as subárvores.

O objetivo primário deste projeto é a construção e a simulação de uma NoC SPIN totalmente funcional, atendendo às restrições arquiteturais de uma topologia *Fat-Tree* de dois níveis para oito *hosts* e oito roteadores. Secundariamente, o projeto buscou:

- garantir a correta interligação dos enlaces UP e DOWN entre as subárvores, permitindo a comunicação cruzada entre clusters de roteadores folha;
- modelar os *handshakes* de transmissão (TX) e recepção (RX) por meio dos sinais *valid* e *ready*, respeitando o sincronismo do relógio e evitando condições de corrida (*race conditions*) inerentes aos *delta-cycles* do simulador SystemC;
- implementar uma lógica de roteamento puramente aritmética, derivada da estrutura em árvore (divisão de índices e comparação de identificadores), sem o uso de tabelas estáticas ou mapeamentos *hardcoded*.

II. REFERENCIAL TEÓRICO

A. Redes em Chip (NoC)

Uma Rede em Chip é uma arquitetura de comunicação que aplica os princípios das redes de computadores ao domínio intrachip. É composta por três elementos fundamentais: os nós de processamento (*hosts*), os roteadores e os enlaces de comunicação. As mensagens trocadas entre *hosts* são subdivididas

em unidades menores denominadas *flits* (*flow control units*), que trafegam pelos roteadores até atingir o destino [1]. O comportamento de cada roteador é definido pelo seu algoritmo de roteamento, pelo mecanismo de arbitragem e pelo controle de fluxo adotado, sendo o esquema *wormhole* e o controle baseado em créditos os mais comuns na literatura [5].

B. Topologia Fat-Tree e Arquitetura SPIN

A *Fat-Tree*, proposta por Leiserson [4], é uma topologia hierárquica em árvore na qual a largura de banda dos enlaces aumenta conforme os nós se aproximam da raiz, eliminando o gargalo típico das árvores convencionais e garantindo múltiplos caminhos paralelos entre os nós. A arquitetura SPIN adota essa topologia para minimizar o congestionamento na comunicação global do chip [3]. Na variante de dois níveis aqui implementada, cada roteador folha conecta-se a dois roteadores raiz por enlaces distintos (UP_0 e UP_1), o que garante redundância de caminhos e distribui a carga de tráfego ascendente.

C. Mecanismos de Comunicação e Controle de Fluxo

O controle de fluxo entre roteadores adjacentes é realizado pelo protocolo de *handshake valid/ready*, amplamente utilizado em projetos de *hardware* digital [6]. O transmissor sinaliza a disponibilidade de um dado por meio do sinal *valid*; o receptor confirma a capacidade de absorção por meio do sinal *ready*. A transferência efetiva ocorre apenas quando ambos os sinais estão simultaneamente em nível alto na borda positiva do relógio, o que opera em nível de enlace e assegura que um roteador só envie dados se o roteador subsequente possuir espaço disponível no *buffer*, eliminando a possibilidade de perda ou duplicação de *flits*.

D. Algoritmo de Roteamento SPIN

O algoritmo de roteamento implementado é determinístico e inteiramente aritmético, dispensando tabelas estáticas. Ele opera de forma diferente conforme o tipo do roteador:

- **Roteadores Folha (IDs 0–3):** a partir do identificador de destino do *flit* ($dest_id$), calcula-se a folha de destino por $dest_leaf = \lfloor dest_id / 2 \rfloor$. Se $dest_leaf$ for igual ao identificador do próprio roteador, o destino é local: o *flit* é encaminhado para LOCAL_0 (destinos pares) ou LOCAL_1 (destinos ímpares). Caso contrário, o tráfego deve subir à raiz, sendo a porta UP_0 ou UP_1 selecionada de acordo com qual dos dois roteadores raiz conectados está topologicamente ligado à folha de destino (Seção III-A).
- **Roteadores Raiz (IDs 4–7):** encaminham o *flit* para a subárvore correta com base em $dest_leaf = \lfloor dest_id / 2 \rfloor$. Destinos cuja folha seja a de menor índice entre as duas folhas ligadas ao roteador seguem para DOWN_0; a folha remanescente é alcançada por DOWN_1.

Essa lógica é puramente matemática (divisões e comparações de índices), o que permite escalar o algoritmo para topologias maiores sem necessidade de vetores ou matrizes de posições *hardcoded*. O fluxograma da Figura 1 sintetiza a árvore de decisão do módulo *RoutingLogic*.

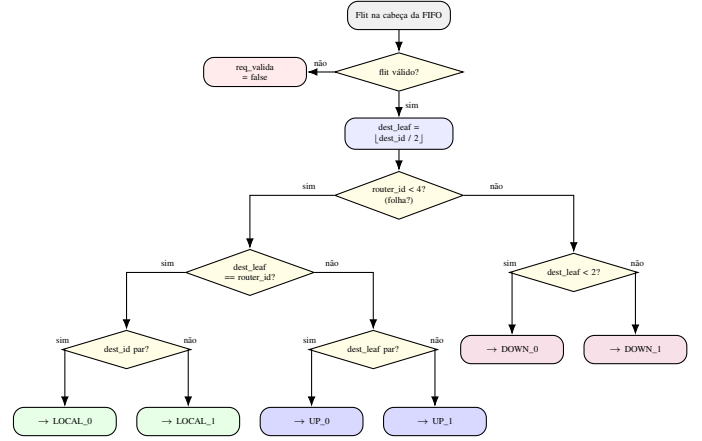


Figura 1. Fluxograma do algoritmo SPIN implementado no módulo *RoutingLogic*: roteadores folha verificam se o destino é local antes de escalar a hierarquia; roteadores raiz selecionam a subárvore de descida com base no índice $dest_leaf$.

E. Arbitragem Round-Robin

O árbitro *Round-Robin* com ponteiro circular é o mecanismo de arbitragem mais utilizado em NoCs por garantir ausência de *starvation*: nenhuma porta de entrada fica indefinidamente sem acesso ao recurso compartilhado [1]. A cada ciclo de relógio em que ocorre uma concessão, o árbitro avança seu ponteiro de prioridade para a porta seguinte à última vencedora, assegurando justiça estatística entre todos os concorrentes pela mesma porta de saída.

III. MODELO IMPLEMENTADO

O projeto foi codificado em SystemC para que o comportamento descrito permanecesse fiel a uma modelagem em nível de transferência de registradores (RTL), simulando de forma síncrona o comportamento de *hardware* real.

A. Visão Geral da Topologia

A rede interconecta oito roteadores, divididos em dois grupos. Os roteadores de IDs 0 a 3 são roteadores folha, cada um conectado a dois *hosts* locais pelas portas LOCAL_0 e LOCAL_1: R0 a {H0, H1}, R1 a {H2, H3}, R2 a {H4, H5} e R3 a {H6, H7}. Os roteadores de IDs 4 a 7 são roteadores raiz, responsáveis por interligar as subárvores. Cada roteador possui até seis portas físicas, identificadas pelo enumerador *PortID* em *noc_pkg.h*: LOCAL_0, LOCAL_1, UP_0, UP_1, DOWN_0 e DOWN_1. Roteadores folha utilizam apenas as portas LOCAL e UP; roteadores raiz utilizam apenas as portas DOWN. As portas remanescentes são conectadas a sinais *dummy*, inicializados explicitamente para evitar avisos de porta desconectada no simulador.

A Tabela I descreve os oito enlaces bidirecionais que interligam folhas e raízes, e a Figura 2 ilustra a topologia resultante: cada roteador folha conecta-se a exatamente dois roteadores raiz distintos, e cada roteador raiz conecta-se a exatamente duas folhas distintas, formando uma malha de redundância que evita a fragmentação da rede em subárvores isoladas.

Tabela I
LINKS BIDIRECIONAIS DA TOPOLOGIA FAT-TREE

Roteador Folha	Roteador Raiz
R0 UP_0 ↔ R4 DOWN_0	R2 UP_0 ↔ R4 DOWN_1
R0 UP_1 ↔ R5 DOWN_0	R3 UP_0 ↔ R5 DOWN_1
R1 UP_0 ↔ R6 DOWN_0	R2 UP_1 ↔ R6 DOWN_1
R1 UP_1 ↔ R7 DOWN_0	R3 UP_1 ↔ R7 DOWN_1

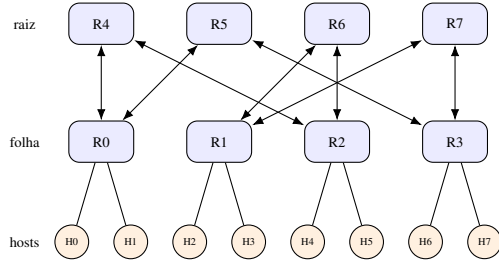


Figura 2. Topologia Fat-Tree de dois níveis com 8 roteadores e 8 hosts.

B. Estrutura do Flit

A unidade mínima de transferência, o *flit*, é definida pela estrutura `Flit` em `noc_pkg.h` e contém os campos `src_id` (identificador do *host* de origem), `dest_id` (identificador do *host* de destino), `payload` (dados úteis transmitidos), `timestamp` (instante de criação, utilizado no cálculo de latência) e `valido` (flag booleana que distingue *flits* reais de *flits* vazios). O operador de igualdade (`==`) foi implementado explicitamente para compatibilidade com `sc_signal<Flit>`, cujo mecanismo de detecção de eventos do SystemC o exige.

C. Componentes do Roteador

Cada roteador é instanciado pela classe `Router` e compõe-se hierarquicamente de quatro tipos de submódulo, cujas interconexões são ilustradas na Figura 3:

1) **FifoBuffer:** Implementado em `fifo_buffer.h/.cpp`, é um *buffer* de entrada por porta, com capacidade fixa de quatro *flits* (`CAPACIDADE_FIFO = 4`). Opera de forma síncrona ao relógio, com interface *valid/ready* tanto na entrada (lado produtor) quanto na saída (lado consumidor). Toda a lógica de controle e atualização de estado foi unificada em um único processo (`SC_METHOD`), avaliando as requisições de leitura e escrita e atualizando as saídas no mesmo pulso positivo do relógio. Essa decisão de projeto eliminou duplicações de *flits* e suprimiu *race conditions* entre *delta-cycles* do simulador.

2) **RoutingLogic:** Implementado em `routing_logic.h/.cpp`, calcula a porta de saída para o *flit* na cabeça da FIFO, conforme o algoritmo descrito na Seção II-D. Opera como um `SC_METHOD` combinacional, sensível a `flit_in` e `flit_valid`. O resultado é um sinal `porta_saida` e uma *flag* `req_valida` destinados ao árbitro.

3) **Arbiter:** Implementado em `arbiter.h/.cpp`, é um árbitro *Round-Robin* com ponteiro circular. A cada borda

positiva do relógio, varre circularmente as seis requisições de entrada (`req_in[0..5]`) a partir do ponteiro de prioridade e concede o acesso (`grant_out`) à primeira porta ativa encontrada, avançando em seguida o ponteiro para a porta seguinte.

4) **Crossbar:** Implementado em `crossbar.h/.cpp`, é uma matriz de comutação 6×6 . Para cada porta de saída j , verifica o *grant* emitido pelo árbitro correspondente e conecta dinamicamente a FIFO de entrada vencedora à saída, propagando simultaneamente o sinal de *ready* (*backpressure*) de volta à FIFO de origem. Opera como um `SC_METHOD` combinacional sensível a todas as entradas.

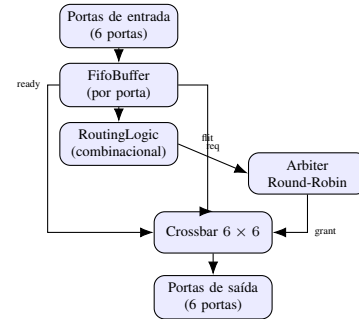


Figura 3. Arquitetura interna do roteador SPIN: o *flit* percorre Entrada → FIFO → RoutingLogic → Arbiter → Crossbar → Saída.

D. Interconexão Interna do Roteador

A classe `Router` instancia e interconecta os submódulos por meio de sinais internos. O fluxo de dados percorre o seguinte caminho: (1) o *flit* chega pela porta física `flit_in[i]` e é armazenado no `FifoBuffer` da porta i ; (2) a `RoutingLogic` da porta i consulta a cabeça da FIFO e calcula a porta de saída j ; (3) um método auxiliar `remapeia_requisicoes` converte o par (i, j) em uma requisição `sig_req[j][i]` para o árbitro da saída j ; (4) o `Arbiter` da saída j concede o acesso com `sig_grant[j] = i`; (5) a `Crossbar` copia o *flit* de `sig_fifo_dout[i]` para `sig_xbar_out[j]`, propagado externamente pela função `conecta_saidas`. A Figura 4 detalha esse percurso sob a perspectiva do ciclo de vida completo de um *flit* na rede.

E. Top-Level e Testbench

O módulo `SpinNocTop` (`spin_noc_top.h`) instancia os oito roteadores, realiza todas as conexões da topologia por meio da função auxiliar `conecta_link` e expõe ao *testbench* uma interface de oito *hosts*: os pares $(tb_in[i], tb_valid_in[i], tb_ready_out[i])$ para injeção de tráfego e $(tb_out[i], tb_valid_out[i], tb_ready_in[i])$ para recepção.

O `SpinNocTb` (`spin_noc_tb.h/.cpp`) divide suas responsabilidades em duas *threads* independentes, cada uma com escrita exclusiva sobre um conjunto de sinais, evitando erros de *driver* duplicado:

- **gerador_trafego:** único escritor dos sinais `rst`, `tb_in[]` e `tb_valid_in[]`. Injeta os três pacotes

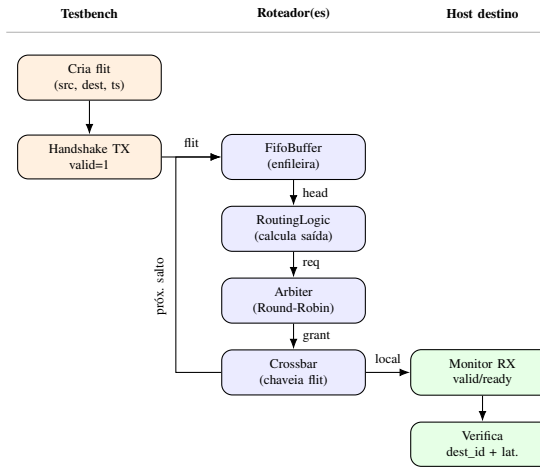


Figura 4. Ciclo de vida de um *flit*: da injeção pelo *testbench* à verificação no *host* de destino, passando pelos submódulos de cada roteador intermediário.

de teste em sequência, aguardando o *handshake* TX entre cada injeção.

- **monitor_recepcao:** único escritor do sinal `tb_ready_in[]`. Monitora continuamente as oito interfaces de saída, captura os *flits* recebidos, calcula a latência e verifica a integridade do campo `dest_id`.

IV. RESULTADOS E SIMULAÇÕES

A. Ambiente de Simulação

A simulação foi executada em SystemC com relógio de 10 ns (*duty cycle* de 50%) durante 2000 ns (200 ciclos). O *reset* foi mantido ativo nos três primeiros ciclos (30 ns) para garantir a inicialização determinística de todos os submódulos, sendo desativado em 20 ns; a NoC entrou em modo operacional em seguida. Um arquivo VCD (`simulation_waveform.vcd`) foi gerado para visualização das formas de onda no GTKWave, contendo os sinais das interfaces dos *hosts* 0, 1, 2, 5 e 7. A Figura ?? ilustra o comportamento esperado do *handshake*: o sinal `valid` é levantado pelo transmissor e somente é desativado após a detecção simultânea de `ready` em nível alto na borda de subida do relógio, sem ocorrência de *glitches* provenientes de *delta-cycles*.

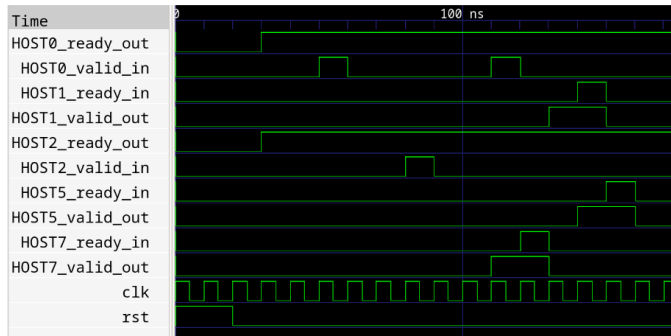


Figura 5. Fluxo de handshake valid/ready sem anomalias (GTKWave).

Três cenários de teste foram implementados para cobrir os extremos do espaço de latência da topologia, conforme a Tabela I: o pior caso, exigindo o número máximo de três saltos entre subárvores totalmente distintas; o cruzamento de subárvores, também com três saltos, porém entre um par diferente de folhas; e o melhor caso, com destino local ao mesmo roteador folha.

B. Cenário 1 — Pior Caso: HOST 0 → HOST 7

O primeiro pacote (*payload* 0xAB) foi injetado por HOST 0 em 50 ns e recebido por HOST 7 em 130 ns, resultando em latência total de 80 ns (8 ciclos de relógio). HOST 0 está conectado ao roteador folha R0, e HOST 7 ao roteador folha R3. Segundo a Tabela I, o único roteador raiz comum às folhas R0 e R3 é R5; o caminho percorrido foi, portanto, R0 → R5 → R3 → HOST 7. O *handshake* TX foi confirmado em 60 ns, demonstrando que a FIFO do roteador R0 aceitou o *flit* em apenas um ciclo após a injeção.

```

@50 ns [TB] Injetando pacote:
  Descricao: PIOR CASO: HOST 0 -> HOST 7
  Payload : 0xAB
@60 ns [TB] Handshake TX confirmado
* PACOTE RECEBIDO - HOST DESTINO: 7
* Tempo de recepcao : 130 ns
* Latencia total : 80 ns
* [OK] Integridade verificada

```

Listing 1. Log do Cenário 1 (pior caso)

C. Cenário 2 — Cruzamento de Subárvores: HOST 2 → HOST 5

O segundo pacote (*payload* 0xCD) foi injetado por HOST 2 em 80 ns e recebido por HOST 5 em 160 ns, resultando também em latência de 80 ns. HOST 2 está conectado a R1, e HOST 5 a R2; o único roteador raiz comum a essas duas folhas é R6, de modo que o caminho percorrido foi R1 → R6 → R2 → HOST 5. A latência idêntica à do Cenário 1 é esperada, pois ambos os percursos envolvem três saltos em uma *Fat-Tree* de dois níveis com FIFOs síncronas de mesma profundidade.

```

@80 ns [TB] Injetando pacote:
  Descricao: CRUZAMENTO: HOST 2 -> HOST 5
  Payload : 0xCD
@90 ns [TB] Handshake TX confirmado
* PACOTE RECEBIDO - HOST DESTINO: 5
* Tempo de recepcao : 160 ns
* Latencia total : 80 ns
* [OK] Integridade verificada

```

Listing 2. Log do Cenário 2 (cruzamento)

D. Cenário 3 — Melhor Caso: HOST 0 → HOST 1

O terceiro pacote (*payload* 0xEF) foi injetado por HOST 0 em 110 ns e recebido por HOST 1 em 150 ns, resultando em latência de 40 ns (4 ciclos). Como HOST 0 e HOST 1 estão ambos conectados ao roteador R0 (pelas portas LOCAL_0 e LOCAL_1, respectivamente), o *flit* não necessita escalar a hierarquia da *Fat-Tree*: a *RoutingLogic* de R0 detecta imediatamente que $\lfloor \text{dest_id}/2 \rfloor = \text{router_id}$ e encaminha o *flit* diretamente para LOCAL_1.

```
@110 ns [TB] Injetando pacote:
  Descricao: MELHOR CASO: HOST 0 -> HOST 1
  Payload : 0xEF
@120 ns [TB] Handshake TX confirmado
* PACOTE RECEBIDO - HOST DESTINO: 1
* Tempo de recepcão : 150 ns
* Latência total : 40 ns
* [OK] Integridade verificada
```

Listing 3. Log do Cenário 3 (melhor caso)

E. Análise de Latência e Integridade

A Tabela II sumariza os resultados dos três cenários. Em todos eles, o pacote foi recebido pelo *host* de destino correto, com o campo *dest_id* verificado pelo *monitor*, e nenhum erro de roteamento, perda ou duplicação de *flit* foi registrado.

Tabela II
RESUMO DOS RESULTADOS DE SIMULAÇÃO

Cenário	Orig.	Dest.	Saltos	Lat.	Status
Pior caso	H0	H7	3	80 ns	OK
Cruzamento	H2	H5	3	80 ns	OK
Melhor caso	H0	H1	0	40 ns	OK

A diferença de latência entre os cenários de três saltos e o cenário local (80ns contra 40ns) reflete diretamente o número de roteadores atravessados: cada roteador adicional introduz um ciclo de processamento na FIFO de entrada e um ciclo de arbitragem e chaveamento. Considerando o relógio de 10 ns, os 40 ns de diferença correspondem a exatamente quatro ciclos adicionais (dois roteadores intermediários, com dois ciclos cada), o que é consistente com o modelo de *pipeline* da arquitetura implementada.

V. CONCLUSÕES

A implementação da Rede em Chip SPIN em SystemC cumpriu as exigências da disciplina de Organização de Computadores e demonstrou a viabilidade de modelar, de forma síncrona e fiel ao nível RTL, uma topologia *Fat-Tree* de dois níveis com oito roteadores. A divisão hierárquica do roteador em submódulos independentes (*FifoBuffer*, *RoutingLogic*, *Arbiter* e *Crossbar*) mostrou-se uma estratégia eficaz tanto para a organização do código quanto para a depuração isolada de cada componente.

Três conclusões técnicas centrais emergem do projeto:

- 1) A correta interligação dos enlaces UP e DOWN, descrita na Tabela I, garantiu que as rotas de cruzamento entre subárvores distintas pudessem ser resolvidas sem perda de pacotes, evitando a fragmentação da rede em ilhas isoladas.
- 2) A unificação das atualizações de estado e de saída dos *FifoBuffers* em um único processo síncrono eliminou de forma absoluta as inconsistências e os *loops* decorrentes de falhas de concorrência entre *delta-cycles* do SystemC.

- 3) A adoção de um algoritmo de roteamento puramente aritmético, derivado de divisões e comparações de identificadores pares/ímpares, resultou em uma arquitetura escalável: o mesmo algoritmo poderia, em princípio, operar sobre topologias maiores sem demandar tabelas ou matrizes de posições *hardcoded*.

Por fim, todos os pacotes disparados — sob cenário de pior caso, tráfego cruzado ou destino local — alcançaram seus destinos com integridade temporal e relacional completa, validando o conjunto de algoritmo de roteamento, mecanismo de arbitragem e controle de fluxo implementados.

A. Dificuldades Encontradas

Tipagem do SystemC e portas sem driver: o rigor do sistema de tipos do SystemC exigiu cuidado na definição dos sinais intermediários. A ausência de inicialização dos sinais *sig_dummy_**, utilizados para conectar portas fisicamente inexistentes dos roteadores folha e raiz, poderia gerar avisos de porta não conectada; isso foi resolvido pela inicialização explícita de todos os sinais *dummy* e pela verificação sistemática de que cada *sc_signal* possuísse exatamente um *driver*.

Determinismo em processos concorrentes: a separação das *threads* gerador_trafego e monitor_recepcão no *testbench*, cada uma com responsabilidade exclusiva sobre um conjunto de sinais de saída, foi essencial para evitar erros de *driver* duplicado (E115); sem essa separação, dois processos escrevendo simultaneamente no mesmo *sc_signal<bool>* gerariam comportamento não determinístico.

Protocolo de handshake no monitor: a implementação correta do protocolo *valid/ready* no monitor exigiu atenção ao sincronismo: o sinal *ready_in* deve ser levantado apenas após a detecção de *valid_out* em nível alto, retornando a zero imediatamente após a leitura do *flit*, evitando dupla captura do mesmo dado em ciclos consecutivos.

B. Trabalhos Futuros

A presente implementação pode ser estendida em diversas direções. A adoção de pacotes multi-*flit* (com *flits* de cabeçalho e de dados separados) aproximaria o modelo de NoCs reais. A implementação de controle de fluxo por créditos (*credit-based*) substituiria o *backpressure* simples por um mecanismo mais robusto em topologias de maior escala. Os três cenários aqui simulados cobrem os extremos de latência da topologia, mas não esgotam todos os pares origem-destino possíveis; como trabalho futuro, recomenda-se uma validação exaustiva de todas as combinações *host-host*, de modo a confirmar a alcançabilidade completa entre todas as folhas pela malha de enlaces raiz. Por fim, o dimensionamento da rede para 16 ou 32 roteadores permitiria avaliar a escalabilidade do algoritmo de roteamento e do árbitro *Round-Robin* sob cargas de tráfego mais elevadas.

REFERÊNCIAS

- [1] W. J. Dally e B. Towles, "Route packets, not wires: On-chip interconnection networks," in *Proc. 38th Design Automation Conference (DAC)*, Las Vegas, NV, EUA, 2001, pp. 684–689.

- [2] L. Benini e G. De Micheli, "Networks on chips: A new SoC paradigm," *IEEE Computer*, vol. 35, n.º 1, pp. 70–78, jan. 2002.
- [3] A. Adriahtenaina, H. Charlery, A. Greiner, L. Mortiez e C. A. Zeferino, "SPIN: A scalable, packet switched, on-chip micro-network," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, Munich, Alemanha, 2003, pp. 70–73.
- [4] C. E. Leiserson, "Fat-trees: Universal networks for hardware-efficient supercomputing," *IEEE Transactions on Computers*, vol. C-34, n.º 10, pp. 892–901, out. 1985.
- [5] S. Kumar *et al.*, "A network on chip architecture and design methodology," in *Proc. IEEE Computer Society Annual Symposium on VLSI*, Pittsburgh, PA, EUA, 2002, pp. 117–124.
- [6] N. H. E. Weste e D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4.ª ed. Boston, MA, EUA: Addison-Wesley, 2011.
- [7] M. E. Kreutz e C. Zeferino, "Redes-em-Chip: Conceitos fundamentais e mecanismos de comunicação," Material de aula, DIM0129 — Organização de Computadores, Universidade Federal do Rio Grande do Norte, 2026.